

Workshop Setup Guide — 2026 SE&ATN Symposium

AI-Driven Bayesian Templates for Hubbard’s AIE in Engineering Decisions Under Uncertainty

Requirements At A Glance

- **Python 3.11+** via Miniconda/Anaconda (recommended) or any existing Python installation
 - **One API key:** OpenAI personal key, OR temporary workshop key from facilitator, OR Ollama locally (no key)
 - **Disk space:** ~2 GB for conda environment with PyMC/ArviZ
-

Files in This Package

File	Purpose
llm_bayesian_reliability_tool_SE-ATN-2026.ipynb	Main tutorial notebook
environment.yml	Reproducible conda environment
requirements.txt	pip fallback (all dependencies)
.env.example	Credential template — copy to <code>.env</code> and fill in
install_before_tutorial.ps1	Windows PowerShell one-click installer
install_before_tutorial.sh	Mac/Linux one-click installer
QUICKSTART.md	Participant quick-start (recommended first read)
WORKSHOP_SETUP.md	This full setup guide

Facilitator Pre-Workshop Checklist

- ☐ Dry-run the full notebook on a clean environment the day before
 - ☐ Set a usage budget on any temporary API keys (OpenAI dashboard)
 - ☐ Share the download link for this package at least 2 days before the session
 - ☐ Ask participants to complete steps 1–3 of QUICKSTART.md before arriving
 - ☐ Have Offline Mode verified as a fallback (the notebook runs without any API key)
 - ☐ Confirm Jupyter Lab launches and the `aie-workshop` kernel appears
-

Participant Setup

Option A — Conda Environment (Primary, Most Reliable)

```
conda env create -f environment.yml
conda activate aie-workshop
python -m ipykernel install --user --name aie-workshop --display-name "Python (aie-workshop)"
jupyter lab
```

Select kernel: **Python (aie-workshop)**

Why conda? PyMC and ArviZ require compiled C extensions. Conda installs pre-built binaries and avoids needing a C compiler on Windows.

Option B — pip with Existing Python 3.11+

```
pip install -r requirements.txt
jupyter lab
```

Option C — Windows One-Click Install

Right-click `install_before_tutorial.ps1` → **Run with PowerShell**
(or double-click `install_before_tutorial.bat`)

Option D — Local Model via Ollama (No API Key Required)

1. Download Ollama: <https://ollama.com/download>
2. After install, open a terminal:

```
ollama pull llama3.1
```

3. In the notebook LLM Configuration cell, set:

```
PROVIDER = "ollama"           # (edit the cell directly)
BASE_URL = "http://localhost:11434/v1"
MODEL = "llama3.1"
API_KEY = "ollama"           # any non-empty string
```

API Key Setup Options

Option 1 — Personal OpenAI Key (Recommended for Participants)

1. Sign in at <https://platform.openai.com/api-keys>
2. Create a new key (costs <\$0.01 for this tutorial with gpt-4o-mini)
3. Set it before launching Jupyter:

Windows PowerShell:

```
$env:LLM_PROVIDER = "openai"
$env:LLM_MODEL     = "gpt-4o-mini"
$env:OPENAI_API_KEY = "sk-your-key-here"
```

Mac/Linux Terminal:

```
export LLM_PROVIDER=openai
export LLM_MODEL=gpt-4o-mini
export OPENAI_API_KEY=sk-your-key-here
```

Or paste directly into the notebook's LLM Configuration cell (easiest for participants):

```
API_KEY = "sk-your-key-here"  # replace this line
```

Option 2 — Facilitator-Provided Workshop Key

The facilitator will share a temporary key with rate limits. Treat it exactly like a personal key above. Do not share it outside the session. Facilitator: revoke the key immediately after the workshop ends.

Option 3 — Offline / No-Key Fallback

The notebook includes a pre-loaded OFFLINE_JSON example. To use it:

In the notebook, find the cell that calls `parse_problem_to_json(...)` and instead run:

```
bayesian_model_json = OFFLINE_JSON  # uncomment this line
```

All downstream cells (validation, DAG, Bayesian model, VoI) work in offline mode.

Verifying Your Key is Set (Without Printing It)

```
if ($env:OPENAI_API_KEY) { "Key is set" } else { "Key NOT set" }
```

Notebook Run Order

Cell	Purpose
Cell 1	Title and tutorial overview (markdown, no output)
Cell 2	Section headers for AIE + Bayesian background (markdown)
Cell 3	Auto-install missing packages (safe to re-run)
Cell 4	Core imports
Cell 5	LLM Configuration (set your API key here)
Cell 6	API connection check

Cell	Purpose
Cell 7	System prompt + <code>parse_problem_to_json()</code> function
Cell 8	Define your problem description (edit YOUR TURN block)
Cell 9+	Call LLM, validate JSON, build Bayesian model, VoI, Monte Carlo

Important: Run cells top to bottom. Do not skip the auto-install cell.

Common Errors and Fixes

Error	Cause	Fix
<code>ModuleNotFoundError: No module named 'pymc'</code>	Package not installed	Re-run Cell 3 or <code>pip install -r requirements.txt</code>
<code>AuthenticationError</code>	Invalid or missing API key	Check key starts with <code>sk-</code> ; no spaces
<code>RateLimitError</code>	API quota exceeded	Wait 60s; use <code>gpt-4o-mini</code> (cheaper); switch to Offline
Kernel not listed	Kernel not registered	<code>python -m ipykernel install --user --name aie-workshop</code>
PyMC sampling hangs	Normal on first run (NUTS warmup)	Wait 30–60s; first sample is always slower
Windows: pymc install fails via pip	Missing C compiler	Use conda (Option A)

Environment Check Command (Quick Preflight)

Run this in a terminal after activating your environment:

```
python -c "
import numpy, scipy, pandas, matplotlib, seaborn, networkx
import openai, pymc, arviz, ipywidgets
print('All packages OK')
print(f'PyMC: {pymc.__version__}  ArviZ: {arviz.__version__}')
"
```

For Facilitators: Distributing This Package

1. Zip the entire SE-ATN-2026/ folder: `SE-ATN-2026-tutorial.zip`
2. Upload to a file sharing link (OneDrive, Google Drive, GitHub Release, etc.)
3. Share the link in session announcements and the day-of slide
4. Recommend participants download and install 1–2 days ahead of the session